TRABAJO FINAL DE GRADO

Gestion de Empresas Colaboradoras para FP Dual

Autor: Luis Angel

Tutora: Elena

Fecha: 08/03/2026

Repositorio: https://github.com/lmendez861/TFG-Agora

Indice

1. Portada
2. Agradecimientos
3. Indice
4. Resumen
5. Antecedentes / Introduccion
6. Objetivos y alcance
7. Definiciones
8. Notaciones y simbolos
9. Desarrollo del trabajo final
10. Conclusiones y recomendaciones
11. Referencias
12. Bibliografia
13. Anexos

Trabajo Final de Grado - Gestion de Empresas Colaboradoras

- Autor: Luis Angel
- Tutora: Elena
- Fecha de revision: 08/03/2026
- Repositorio: https://github.com/lmendez861/TFG-Agora

1. Portada

Titulo: Gestion de Empresas Colaboradoras para FP Dual Autor: Luis Angel Tutora: Elena Fecha: 08/03/2026

2. Agradecimientos

Quiero agradecer a mi tutora Elena su seguimiento, sus observaciones y la orientacion prestada durante el desarrollo de este trabajo. Tambien agradezco al profesorado del ciclo y al centro educativo el contexto real aportado para enfocar el proyecto hacia una necesidad concreta y utilizable. Por ultimo, agradezco a mi entorno personal el apoyo y la constancia durante el proceso de analisis, implementacion, pruebas y redaccion final de la memoria.

3. Indice

En la maquetacion final se incluira indice general, indice de tablas e indice de figuras.

4. Resumen

4.1 Resumen (ES)

Este proyecto desarrolla una plataforma web para gestionar empresas colaboradoras, convenios, estudiantes, tutores y solicitudes externas en un entorno de FP Dual. La solucion combina una API en Symfony, un panel interno en React/TypeScript y un portal externo para empresas. El sistema ya permite CRUD de entidades principales, seguimiento documental, workflow de convenios, mensajeria asociada a solicitudes y control de acceso por roles. La validacion tecnica actual confirma 47 pruebas backend superadas y compilacion correcta de ambos frontends.

Palabras clave: FP Dual, empresas colaboradoras, convenios, Symfony, React, gestion academica.

4.2 Summary (EN)

This project delivers a web platform to manage partner companies, agreements, students, mentors, and external registration requests for dual training. The solution combines a Symfony API, an internal React/TypeScript dashboard, and an external company portal. The system already supports CRUD operations for the main entities, document handling, agreement workflow, request messaging, and role-based access control. Current technical validation confirms 47 passing backend tests and successful production builds for both frontends.

Keywords: dual training, partner companies, agreements, Symfony, React, academic management.

5. Antecedentes / Introduccion

- La gestion inicial de empresas colaboradoras, convenios y practicas se hacia con hojas de calculo, correos y documentos dispersos.
- Ese enfoque generaba duplicidades, falta de trazabilidad, poca visibilidad de estados y dependencia de conocimiento manual.
- El proyecto Agora original se reutilizo solo como base tecnica. La logica previa de chats y bots se archivo en `legacy/`.
- El nuevo alcance se centra en un problema real del centro: registrar empresas, aprobar solicitudes, formalizar convenios y controlar asignaciones y seguimientos.
- El sistema esta orientado a coordinacion de practicas, tutores academicos, tutores profesionales y empresas interesadas en colaborar.
- La propuesta se alinea con la necesidad de profesionalizar la gestion de la FP Dual, donde la coordinacion entre centro y empresa exige trazabilidad documental, control de estados y una comunicacion mas estructurada.

6. Objetivos y alcance

6.1 Objetivo general

Disenar e implantar una aplicacion web que centralice la gestion de empresas colaboradoras y practicas formativas, unificando informacion operativa, control documental y seguimiento de estados en una unica plataforma.

6.2 Objetivos especificos

1. Digitalizar el ciclo de vida de empresa, convenio, estudiante, tutor y asignacion.
2. Habilitar un flujo publico de solicitud de empresa con verificacion y aprobacion interna.
3. Ofrecer un panel interno con KPI, tablas de gestion, detalle de entidades y acciones CRUD.
4. Incorporar workflow documental y alertas para convenios y empresas.
5. Proteger la API y el panel con autenticacion y jerarquia de roles.

6.3 Alcance

- Dentro del alcance: panel interno, portal externo, API REST, base de datos relacional, usuarios demo, solicitudes de empresas, mensajeria vinculada, documentos, checklist, alertas y exportacion CSV de listados operativos.
- Fuera del alcance actual: firma electronica avanzada, integracion con ERP/SGA, exportacion PDF maquetada, almacenamiento documental en nube y pruebas E2E completas.

7. Definiciones

- Empresa colaboradora: organizacion que puede acoger estudiantes del centro.
- Convenio: acuerdo que formaliza la colaboracion entre el centro y una empresa.
- Asignacion: relacion operativa entre estudiante, convenio, empresa y tutores.
- Seguimiento: registro de reuniones, incidencias o acciones de control sobre una asignacion.
- Solicitud de empresa: alta inicial enviada por una empresa desde el portal externo antes de ser aprobada.

8. Notaciones y simbolos

- API: Application Programming Interface.
- CRUD: Create, Read, Update, Delete.
- FP Dual: Formacion Profesional Dual.
- KPI: Key Performance Indicator.
- RGPD: Reglamento General de Proteccion de Datos.
- SPA: Single Page Application.

9. Desarrollo del trabajo final

9.1 Analisis de requisitos

- Actores principales: coordinador o administrador, tutor academico, tutor profesional, estudiante y empresa externa.
- Casos de uso principales: consultar KPI, registrar empresas, crear y actualizar convenios, gestionar estudiantes y asignaciones, revisar solicitudes externas, adjuntar documentos, intercambiar mensajes y consultar detalle historico.

- Requisitos funcionales: listados con filtros, operaciones CRUD, validaciones de negocio, workflow de convenios, aprobacion y rechazo de solicitudes, detalle de entidades, notificaciones y consulta de tutores.
- Requisitos no funcionales: interfaz responsive, errores JSON normalizados, seguridad por roles, arquitectura mantenible, base de datos portable y despliegue reproducible en entorno local.

9.1.1 Planificacion y metodologia de trabajo

- Fase 1. Analisis del problema y definicion del alcance real para centrar el proyecto en empresas, convenios, estudiantes, asignaciones y solicitudes externas.
- Fase 2. Diseno tecnico de arquitectura, entidades y relaciones, separando API, panel interno y portal externo para reducir acoplamiento.
- Fase 3. Implementacion iterativa por modulos, validando primero CRUD nucleares y despues funcionalidades de valor como workflow de convenios, mensajeria y exportacion CSV.
- Fase 4. Pruebas tecnicas, correccion de incidencias y preparacion de documentacion de entrega, incluyendo memoria, anexos y guia de demo.
- La metodologia seguida ha sido incremental: cada bloque funcional se ha cerrado con verificacion tecnica antes de ampliar el alcance.

9.2 Diseno de la solucion

- La solucion se divide en tres piezas: backend Symfony 7.3, panel interno React 18 + TypeScript + Vite y portal externo React 19 + TypeScript + Vite.
- El backend expone una API REST protegida bajo `/api`, junto con rutas publicas para registro de empresas y rutas de portal basadas en token.
- El panel interno consume la API mediante un cliente ligero (`frontend/app/src/services/api.ts`) con cabecera Basic automatica desde variables de entorno.
- El portal externo permite registrar empresas, verificar el correo con token y usar un canal de mensajeria asociado a la solicitud.
- La persistencia se resuelve con Doctrine ORM y SQLite por defecto en desarrollo, aunque la configuracion admite otras bases soportadas por Symfony.

9.3 Modelo de datos

- Entidades nucleares: `EmpresaColaboradora`, `Convenio`, `Estudiante`, `TutorAcademico`, `TutorProfesional`, `AsignacionPractica`, `Seguimiento` y `EvaluacionFinal`.
- Entidades de soporte a negocio: `EmpresaSolicitud`, `EmpresaMensaje`, `EmpresaDocumento`, `EmpresaEtiqueta`, `EmpresaNota`, `ConvenioDocumento`, `ConvenioChecklistItem`, `ConvenioAlerta` y `ConvenioWorkflowEvento`.
- Seguridad y acceso: entidad `User` con roles `ROLE_USER`, `ROLE_API` y `ROLE_ADMIN`.
- Relaciones principales: una empresa tiene convenios, contactos, documentos, notas y tutores; un convenio genera asignaciones y eventos de workflow; cada asignacion enlaza estudiante, empresa y tutores, y produce seguimientos y evaluacion final.

9.4 Seguridad y cumplimiento

- La API usa autenticacion HTTP Basic y tambien deja preparado `json_login` para integraciones futuras.
- La jerarquia de seguridad define `ROLE_ADMIN` por encima de `ROLE_API` y `ROLE_USER`.
- Las rutas publicas se limitan al alta de solicitudes y a la confirmacion de correo/token; el resto de recursos de gestion requieren autenticacion.
- Los controladores devuelven respuestas JSON con mensajes de error consistentes y codigos HTTP adecuados.
- En cumplimiento y privacidad, el sistema trabaja con datos minimos necesarios, separa portal publico y panel interno, y deja como mejora futura la auditoria avanzada y la politica definitiva de conservacion documental.

9.5 Diseno de interfaz

- El panel principal se organiza en modulos: dashboard, empresas, convenios, estudiantes, asignaciones, tutores, solicitudes y documentacion.
- La interfaz combina tablas, tarjetas de detalle, chips de estado, modales de formulario, toasts y un centro de notificaciones para solicitudes pendientes.
- La vista de convenios incluye checklist, workflow, documentos y alertas activas o descartadas.
- La vista de empresas concentra KPI, convenios asociados, asignaciones, notas, etiquetas y documentos.
- El portal externo ofrece una experiencia diferenciada y mas simple: formulario de alta, bandeja de verificacion y chat empresa-centro.

9.6 Implementacion

- Backend:

  - Controladores REST para empresas, convenios, estudiantes, asignaciones y tutores. - Controladores especificos para solicitudes de empresa, mensajeria asociada y portal por token. - Fixtures demo con usuarios, empresas, convenios, estudiantes, tutores, asignaciones, checklist, alertas y mensajes.

- Frontend interno:

  - `App.tsx` actua como shell del panel, rutas, dashboard y modulos de detalle. - Formularios modales reutilizables para empresas, convenios, estudiantes y asignaciones. - Cliente API con helpers `apiGet`, `apiPost`, `apiPut`, `PATCH` y soporte para subida de documentos.

- Frontend externo:

  - Landing con formulario publico. - Confirmacion de email mediante token. - Chat simple sobre la solicitud mientras esta en revision.

- Estado actual de implementacion: la funcionalidad principal esta operativa y validada; el panel ya incorpora exportacion CSV de resumenes y listados, y el modulo de convenios persiste workflow, checklist y alertas mediante la API. Los desarrollos abiertos se concentran en endurecimiento productivo, exportacion PDF y automatizacion E2E.

9.7 Pruebas y validacion

- El backend dispone de una suite automatizada con 47 pruebas y 264 aserciones.
- La bateria cubre autenticacion, listado y detalle de empresas, convenios, estudiantes y asignaciones, flujo completo de solicitudes, portal por token, documentos y repositorios.
- El 08/03/2026 se verifico ademas el arranque real de los tres servicios locales:

  - API Symfony en `http://127.0.0.1:8000` - Panel interno en `http://localhost:5173` - Portal externo en `http://localhost:5174`

- Ambos frontends generan build de produccion sin errores con `npm run build`.
- Como limitacion de validacion, todavia no existe una suite E2E de navegador.

9.8 Despliegue y operacion

- En desarrollo local se usa `backend/.env.local` con SQLite y `frontend/app/.env.local` para URL base y credenciales de API.
- El backend puede iniciarse con Symfony CLI o servidor PHP local; los dos frontends se ejecutan con Vite en puertos separados.
- La operacion cotidiana requiere unicamente PHP, Composer, Node.js, npm y la base SQLite local.
- Para una entrega productiva se recomienda separar frontends estaticos, API Symfony y almacenamiento documental, ademas de sustituir Basic auth por un mecanismo mas robusto.
- La guia operativa del proyecto queda recogida en `docs/guia-demo.md`.

9.9 Resultados y metricas (08/03/2026)

- Tres servicios levantados y accesibles de forma concurrente: backend, panel interno y portal externo.
- La llamada autenticada a `/api/empresas` devuelve respuesta valida y datos de demo.
- `php bin/phpunit` finaliza en verde con 47 pruebas y 264 aserciones.
- `npm run build` se completa correctamente en `frontend/app` y `frontend/company-portal`.
- El panel interno permite exportar a CSV el dashboard, empresas, convenios, estudiantes, asignaciones, tutores y solicitudes.
- El flujo funcional mas relevante queda cubierto: solicitud externa, confirmacion por token, aprobacion interna, consulta de detalle, workflow de convenios persistido y gestion documental basica.

9.10 Limitaciones y riesgos

- La autenticacion actual resuelve el entorno academico y de demo, pero no cubre escenarios de SSO, MFA o auditoria avanzada.
- El modulo de asignaciones expone seguimientos y evaluacion final en consulta, pero no incorpora aun un CRUD especifico completo para editar estas piezas desde la API y el panel.
- El modulo de tutores se entrega como consulta operativa con exportacion, pero no como gestion CRUD completa de tutores academicos y profesionales.

- Las entidades principales no disponen todavia de un ciclo completo de archivado o baja logica con endpoints dedicados de cierre de vida.
- El portal externo cubre alta, verificacion y chat, pero no incluye aun un area postaprobacion para empresa ya validada.
- No existe integracion con sistemas corporativos externos ni firma electronica avanzada.
- El almacenamiento de documentos sigue siendo local y debe endurecerse antes de usar datos reales.
- Falta automatizar pruebas E2E del panel y del portal para reducir riesgo de regresion visual o de flujo.
- Las capturas finales y evidencias visuales pueden ampliarse en una revision posterior para una memoria mas visual, aunque el contenido tecnico y de defensa queda ya consolidado.

9.11 Verificacion tecnica (08/03/2026)

- Backend:

  - `php bin/phpunit` -> OK, 47 tests, 264 assertions. - `symfony server:start --no-tls -d --port=8000` -> servicio operativo.

- Frontend interno:

  - `npm run dev -- --host --port 5173 --strictPort` -> servicio operativo. - `npm run build` -> compilacion correcta.

- Portal externo:

  - `npm run dev -- --host --port 5174 --strictPort` -> servicio operativo. - `npm run build` -> compilacion correcta.

- Verificacion HTTP:

  - `GET /api/empresas` con `admin/admin123` -> `200 OK`. - `GET /` en `5173` y `5174` -> `200 OK`.

10. Conclusiones y recomendaciones

- El proyecto ya cumple el objetivo principal de centralizar la gestion de empresas colaboradoras y practicas en una plataforma unica y operativa.
- La separacion entre panel interno, portal externo y API permite escalar funcionalidades sin mezclar responsabilidades.
- El mayor valor aportado al centro es la trazabilidad del ciclo completo: solicitud, aprobacion, convenio, asignacion, seguimiento y documentacion.
- Para una siguiente iteracion se recomienda, por este orden, completar el CRUD de seguimientos y evaluacion final, ampliar la gestion de tutores, incorporar archivado o baja logica de entidades, habilitar un portal postaprobacion para empresas y automatizar pruebas E2E.
- A nivel de explotacion real tambien seria recomendable reforzar almacenamiento documental, seguridad avanzada e integraciones corporativas.

11. Referencias

1. Proyecto TFG Agora. `docs/domain-model.md`.
2. Proyecto TFG Agora. `docs/guia-demo.md`.
3. Symfony. *Symfony Documentation*.
4. Doctrine Project. *Doctrine ORM Documentation*.
5. React Team. *React Documentation*.
6. Parlamento Europeo y Consejo de la Union Europea. *Reglamento (UE) 2016/679*.

12. Bibliografia

- Documentacion oficial de Vite y TypeScript para la construccion de SPA modernas.
- Material docente del modulo de proyecto y guias internas del centro.
- Referencias sobre gestion de practicas, modelado REST y seguridad web.

13. Anexos

Anexo A. Manual de usuario

Referencia principal: `docs/anexo-a-manual-usuario.md`.

Anexo B. Manual tecnico

Referencia principal: `docs/anexo-b-manual-tecnico.md`.

Anexo C. Evidencias de validacion

Referencia principal: `docs/anexo-c-capturas-y-evidencias.md`.

Anexo D. Artefactos de apoyo

Referencias principales:

- `docs/anexo-d-codigo-relevante.md`
- `docs/domain-model.md`
- `docs/refactor-plan.md`

Anexo E. Defensa del proyecto

Referencias principales:

- `docs/anexo-e-defensa.md`
- `docs/guion-defensa.md`
- `docs/preguntas-defensa.md`

Anexo A. Manual de Usuario

1. Objetivo

Este anexo describe el uso funcional de la plataforma "Gestion de Empresas Colaboradoras" desde el punto de vista del usuario interno del centro y del contacto externo de empresa.

2. Perfiles de uso

- `ROLE_ADMIN`: administracion general del panel y acceso completo a recursos API.
- `ROLE_API`: operativa de gestion sobre empresas, convenios, estudiantes, asignaciones y solicitudes.
- Empresa externa: uso del portal publico para registrar su interes, verificar el correo y comunicarse con el centro.

3. Acceso al sistema

3.1 Panel interno

- URL: `http://localhost:5173`
- Credenciales demo:
  - `admin / admin123` - `coordinador / coordinador123`

3.2 Portal externo

- URL: `http://localhost:5174`
- No requiere autenticacion previa para registrar una solicitud.

4. Navegacion principal del panel interno

El panel se estructura en los siguientes modulos:

- Dashboard
- Empresas
- Convenios
- Estudiantes
- Asignaciones
- Tutores
- Solicitudes
- Documentacion

5. Dashboard

Desde el dashboard se puede:

- consultar KPI principales del sistema;
- revisar tarjetas resumen por modulo;
- abrir accesos rapidos a las areas operativas;
- exportar un CSV de resumen con indicadores y analitica.

6. Gestion de empresas

En el modulo de empresas el usuario puede:

- ver el listado general;
- crear una nueva empresa;
- editar una empresa existente;
- consultar la ficha 360;
- revisar convenios asociados;
- revisar asignaciones vinculadas;
- consultar notas, etiquetas y documentos;
- exportar el listado visible a CSV.

7. Gestion de convenios

En el modulo de convenios el usuario puede:

- listar convenios por empresa o estado;
- crear y editar convenios;
- revisar workflow y checklist documental;
- adjuntar documentos de apoyo;
- revisar alertas activas;
- exportar el listado visible a CSV.

8. Gestion de estudiantes

En el modulo de estudiantes el usuario puede:

- listar estudiantes registrados;
- dar de alta y editar fichas;
- revisar estado academico y asignaciones;
- consultar seguimiento resumido;
- exportar el listado visible a CSV.

9. Gestion de asignaciones

En el modulo de asignaciones el usuario puede:

- consultar el pipeline completo;
- filtrar por estado y modalidad;
- crear nuevas asignaciones;
- editar asignaciones existentes;
- abrir la ficha de detalle;
- exportar el listado visible a CSV.

10. Gestion de tutores

En el modulo de tutores el usuario puede:

- consultar tutores academicos;
- consultar tutores profesionales;
- refrescar datos paginados;
- exportar cada tabla a CSV.

11. Solicitudes de empresa

En el modulo de solicitudes el usuario puede:

- revisar nuevas solicitudes enviadas desde el portal;
- aprobar solicitudes verificadas;
- rechazar solicitudes indicando motivo;
- consultar e intercambiar mensajes con la empresa;
- exportar el listado visible a CSV.

12. Portal externo

El flujo del portal externo es:

1. La empresa completa el formulario de solicitud.
2. Recibe un enlace de verificacion por correo.
3. Confirma el token desde la vista de verificacion.
4. Usa el canal de mensajes mientras su solicitud esta en revision.

13. Errores y validaciones comunes

- Si el backend no responde, el panel mostrara mensajes de error y no cargara las colecciones.
- Si faltan credenciales o son incorrectas, la API devolvera error de autenticacion.
- Si un formulario contiene datos invalidos, el modal mostrara el mensaje correspondiente.

14. Recomendaciones de uso

- Refrescar solicitudes antes de aprobar o rechazar.
- Revisar el estado documental de convenios antes de avanzar workflow.
- Exportar CSV como apoyo para revision, seguimiento y defensa del TFG.

Anexo B. Manual Tecnico

1. Arquitectura general

La solucion se compone de tres aplicaciones:

- `backend/`: API Symfony 7.3 con Doctrine ORM y SQLite por defecto.
- `frontend/app/`: panel interno React 18 + TypeScript + Vite.
- `frontend/company-portal/`: portal externo React 19 + TypeScript + Vite.

2. Requisitos locales

- PHP 8.2
- Composer 2.x
- Node.js 18+
- npm
- SQLite
- Symfony CLI opcional

3. Arranque del entorno

3.1 Backend

```
cd backend
composer install
php bin/console doctrine:migrations:migrate --no-interaction
php bin/console doctrine:fixtures:load --no-interaction
symfony server:start --no-tls -d --port=8000
```

3.2 Frontend interno

```
cd frontend/app
npm install
npm run dev -- --host --port 5173 --strictPort
```

3.3 Portal externo

```
cd frontend/company-portal
npm install
```

```
npm run dev -- --host --port 5174 --strictPort
```

4. Variables de entorno relevantes

4.1 Backend

- `DATABASE_URL`
- `MAILER_DSN`
- `APP_MAIL_FROM`

4.2 Frontend interno

- `VITE_API_BASE_URL`
- `VITE_API_USERNAME`
- `VITE_API_PASSWORD`

5. Seguridad

- El backend protege `/api` mediante seguridad Symfony.
- Se usa HTTP Basic y `json_login` queda preparado para evolucion posterior.
- Jerarquia de roles:
  - `ROLE_ADMIN` - `ROLE_API` - `ROLE_USER`

6. Persistencia

La base de datos por defecto es SQLite. El esquema actual se crea a partir de la migracion base:

- `backend/migrations/Version20251113203450.php`

La carga de datos demo se realiza mediante:

- `backend/src/DataFixtures/DemoDominioFixtures.php`

7. Estructura funcional del backend

Controladores principales:

- `backend/src/Controller/Api/EmpresaColaboradoraController.php`
- `backend/src/Controller/Api/ConvenioController.php`
- `backend/src/Controller/Api/EstudianteController.php`
- `backend/src/Controller/Api/AsignacionController.php`
- `backend/src/Controller/Api/TutorAcademicoController.php`
- `backend/src/Controller/Api/TutorProfesionalController.php`
- `backend/src/Controller/Api/EmpresaSolicitudController.php`
- `backend/src/Controller/Api/EmpresaMensajeController.php`

- `backend/src/Controller/RegistroEmpresaController.php`
- `backend/src/Controller/Portal/SolicitudPortalController.php`

8. Estructura funcional del frontend interno

Archivos clave:

- `frontend/app/src/App.tsx`
- `frontend/app/src/components/DataTable.tsx`
- `frontend/app/src/components/EmpresaForm.tsx`
- `frontend/app/src/components/ConvenioForm.tsx`
- `frontend/app/src/components/EstudianteForm.tsx`
- `frontend/app/src/components/AsignacionForm.tsx`
- `frontend/app/src/services/api.ts`
- `frontend/app/src/utils/csv.ts`

9. Exportacion CSV

La exportacion CSV se realiza en cliente mediante:

- `frontend/app/src/utils/csv.ts`

Actualmente se exportan:

- dashboard;
- empresas;
- convenios;
- estudiantes;
- asignaciones;
- tutores;
- solicitudes.

10. Validacion tecnica

Comandos principales:

```
cd backend
php bin/phpunit
```

```
cd frontend/app
npm run build
```

```
cd frontend/company-portal
npm run build
```

11. Riesgos tecnicos abiertos

- no existe suite E2E de navegador;
- la autenticacion del panel sigue basada en credenciales de entorno;
- la salida PDF de la memoria no se genera desde este repositorio;
- el almacenamiento documental sigue siendo local.

Anexo C. Capturas y Evidencias

1. Capturas obligatorias recomendadas

Tomar una captura final de cada una de estas vistas:

1. Dashboard principal con KPI y analitica.
2. Modulo de empresas con listado y ficha 360 abierta.
3. Modulo de convenios con workflow, checklist y alertas.
4. Modulo de estudiantes con ficha academica y asignaciones.
5. Modulo de asignaciones con pipeline y detalle.
6. Modulo de tutores con ambas tablas visibles.
7. Modulo de solicitudes con mensajes y acciones de aprobacion/rechazo.
8. Portal externo con formulario de alta.
9. Portal externo con pantalla de verificacion de token.
10. Portal externo con canal de mensajes.

2. Evidencias tecnicas a incluir en anexos

- salida de `php bin/phpunit`;
- salida de `npm run build` en `frontend/app`;
- salida de `npm run build` en `frontend/company-portal`;
- comprobacion de `GET /api/empresas` con respuesta `200 OK`;
- comprobacion visual de exportacion CSV.

3. Nombre recomendado para las capturas

- `01-dashboard.png`
- `02-empresas.png`
- `03-convenios.png`
- `04-estudiantes.png`
- `05-asignaciones.png`
- `06-tutores.png`
- `07-solicitudes.png`
- `08-portal-alta.png`
- `09-portal-verificacion.png`
- `10-portal-chat.png`

4. Ubicacion sugerida

Guardar las capturas finales en:

- `docs/capturas/`

5. Comprobacion antes de incrustar en la memoria

- verificar que la fecha y hora del sistema no distraigan;
- ocultar credenciales o datos irrelevantes;
- mantener una resolucion suficiente para impresion;
- usar siempre el mismo zoom y ancho de ventana;
- capturar tambien una exportacion CSV descargada como prueba funcional.

Anexo D. Codigo Relevante

1. Backend

Seguridad y autenticacion

- `backend/config/packages/security.yaml`
- `backend/src/Controller/Api/AuthController.php`
- `backend/src/Entity/User.php`

Dominio y persistencia

- `backend/src/Entity/EmpresaColaboradora.php`
- `backend/src/Entity/Convenio.php`
- `backend/src/Entity/Estudiante.php`
- `backend/src/Entity/AsignacionPractica.php`
- `backend/src/Entity/EmpresaSolicitud.php`
- `backend/migrations/Version20251113203450.php`
- `backend/src/DataFixtures/DemoDominioFixtures.php`

Casos de uso principales

- `backend/src/Controller/Api/EmpresaColaboradoraController.php`
- `backend/src/Controller/Api/ConvenioController.php`
- `backend/src/Controller/Api/EstudianteController.php`
- `backend/src/Controller/Api/AsignacionController.php`
- `backend/src/Controller/Api/EmpresaSolicitudController.php`
- `backend/src/Controller/Portal/SolicitudPortalController.php`

2. Frontend interno

Shell principal y modulos

- `frontend/app/src/App.tsx`

Formularios reutilizables

- `frontend/app/src/components/EmpresaForm.tsx`
- `frontend/app/src/components/ConvenioForm.tsx`
- `frontend/app/src/components/EstudianteForm.tsx`
- `frontend/app/src/components/AsignacionForm.tsx`

Infraestructura de UI

- `frontend/app/src/components/DataTable.tsx`
- `frontend/app/src/components/Modal.tsx`
- `frontend/app/src/components/ToastStack.tsx`

Cliente API y exportacion

- `frontend/app/src/services/api.ts`
- `frontend/app/src/utils/csv.ts`

3. Frontend externo

- `frontend/company-portal/src/App.tsx`

4. Suite de pruebas destacada

- `backend/tests/Security/AuthenticationTest.php`
- `backend/tests/Controller/Portal/SolicitudPortalControllerTest.php`
- `backend/tests/Controller/Api/EmpresaSolicitudFlowTest.php`
- `backend/tests/Controller/Api/ConvenioControllerTest.php`
- `backend/tests/Controller/Api/AsignacionControllerTest.php`
- `backend/tests/Controller/Api/EmpresaControllerTest.php`

5. Criterio de seleccion

Este anexo recoge las piezas mas representativas para explicar:

- arquitectura;
- seguridad;
- modelo de dominio;
- flujos de negocio;
- validacion tecnica;
- exportacion funcional.

Anexo E. Defensa del Proyecto

1. Objetivo del anexo

Este anexo recoge el material de apoyo para la exposicion oral del Trabajo Final de Grado. Su funcion es resumir de forma clara el problema abordado, la solucion implementada, los resultados validados y las limitaciones asumidas dentro del alcance academico del proyecto.

2. Mensaje principal de la defensa

La idea central de la presentacion es que el proyecto ya resuelve el ciclo operativo principal de gestion de empresas colaboradoras para FP Dual:

- alta de empresas desde portal externo;
- verificacion mediante token;
- aprobacion interna;
- gestion de empresas, convenios, estudiantes y asignaciones;
- seguimiento documental y exportacion operativa.

3. Guion breve de exposicion

3.1 Apertura

El proyecto surge para resolver una gestion dispersa basada en hojas de calculo, correos y documentos aislados. La propuesta consiste en una plataforma unica que centraliza empresas colaboradoras, convenios y practicas formativas.

3.2 Problema

La gestion previa generaba duplicidades, poca trazabilidad, dependencia del conocimiento manual y escasa visibilidad del estado real de convenios, solicitudes y asignaciones.

3.3 Solucion

La solucion se ha dividido en tres piezas:

1. API REST en Symfony.
2. Panel interno en React y TypeScript.
3. Portal externo para empresas.

3.4 Valor funcional

Las funcionalidades con mas peso en la demo son:

- solicitud externa de empresa;
- verificacion por token;

- aprobacion desde panel interno;
- workflow de convenios con checklist y alertas;
- gestion de entidades principales y exportacion CSV.

3.5 Cierre

La conclusion defendible es que el proyecto ya entrega una solucion operativa y escalable para el centro, aunque deja identificadas mejoras futuras claras y razonables.

4. Preguntas previsibles y linea de respuesta

4.1 Por que esta arquitectura?

Porque separa responsabilidades entre API, panel interno y portal externo, lo que facilita mantenimiento, seguridad y evolucion futura.

4.2 Cual es la aportacion real del proyecto?

Digitalizar un proceso real del centro y convertirlo en una plataforma trazable con mejor control documental y operativo.

4.3 Que queda como trabajo futuro?

CRUD de seguimientos y evaluacion final, gestion completa de tutores, archivado de entidades, portal postaprobacion para empresas, pruebas E2E y seguridad productiva reforzada.

4.4 Por que es valido como entrega final aunque queden mejoras?

Porque el objetivo principal esta cumplido, el flujo clave funciona extremo a extremo y las mejoras pendientes no bloquean la demostracion ni invalidan la solucion entregada.

5. Orden recomendado para la demo

1. Dashboard y KPI.
2. Empresas y ficha 360.
3. Convenios con workflow, checklist y alertas.
4. Asignaciones y relaciones con estudiantes y tutores.
5. Solicitudes de empresa.
6. Portal externo.

6. Mensaje de cierre recomendado

El TFG no se limita a un prototipo visual, sino que entrega una base funcional completa con modelo de datos, API, panel, portal y documentacion. El valor principal es la centralizacion de la gestion y la trazabilidad del ciclo completo entre centro y empresa.

Guion de Defensa del TFG

Objetivo

Este guion esta pensado para una exposicion breve de 5 a 7 minutos centrada en problema, solucion, demo y cierre.

1. Apertura

Buenos dias. Mi Trabajo Final de Grado consiste en el desarrollo de una plataforma para gestionar empresas colaboradoras, convenios y practicas de FP Dual. El objetivo ha sido sustituir una gestion dispersa basada en hojas de calculo, correos y documentos aislados por una herramienta unica, trazable y utilizable por el centro.

2. Problema detectado

La situacion inicial presentaba tres problemas principales:

- informacion repartida en varias fuentes;
- falta de control sobre estados, convenios y seguimiento;
- dependencia del conocimiento manual de quien coordinaba las practicas.

Esto generaba duplicidades, retrasos y poca visibilidad del ciclo completo entre centro, empresa y estudiante.

3. Objetivo y alcance

El objetivo general ha sido crear una aplicacion web que centralice la gestion de empresas colaboradoras y practicas formativas. Dentro del alcance se incluyen empresas, convenios, estudiantes, asignaciones, tutores y solicitudes externas. Quedan fuera de esta version aspectos como firma electronica avanzada, SSO corporativo o una suite E2E completa.

4. Solucion implementada

La solucion se ha dividido en tres partes:

- una API REST en Symfony;
- un panel interno en React y TypeScript para la gestion diaria;
- un portal externo para que las empresas puedan registrarse, verificar su solicitud y comunicarse con el centro.

Esta separacion permite mantener responsabilidades claras y hace el sistema mas escalable.

5. Funcionalidades clave

Las funcionalidades mas relevantes son:

- gestion CRUD de empresas, convenios, estudiantes y asignaciones;
- workflow documental de convenios con checklist y alertas;
- solicitudes externas con verificacion por token y aprobacion interna;
- mensajeria asociada a la solicitud;
- exportacion CSV de dashboard y listados operativos.

6. Demo sugerida

Durante la demostracion voy a seguir este recorrido:

1. Dashboard con KPI y exportacion CSV.
2. Modulo de empresas con ficha 360.
3. Modulo de convenios con workflow, checklist y alertas.
4. Modulo de asignaciones y relaciones con estudiantes y tutores.
5. Modulo de solicitudes.
6. Portal externo con alta, verificacion y chat.

7. Resultados tecnicos

La aplicacion se ha validado con:

- 47 pruebas backend y 264 aserciones en verde;
- compilacion correcta de ambos frontends;
- servicios locales funcionales para backend, panel interno y portal externo.

Ademas, el proyecto queda documentado con memoria, anexos tecnicos, manual de usuario y guia de demo.

8. Limitaciones y mejora futura

Las principales limitaciones actuales son:

- autenticacion mejorable para un entorno productivo;
- falta de exportacion PDF maquetada;
- almacenamiento documental local;
- ausencia de pruebas E2E completas.

Como trabajo futuro, el siguiente paso seria endurecer seguridad, automatizar pruebas y completar la operacion documental.

9. Cierre

En conclusion, el proyecto cumple el objetivo de digitalizar y centralizar la gestion de empresas colaboradoras y practicas. Aporta trazabilidad, mejora operativa y una base tecnica escalable para evolucionar el sistema en futuras iteraciones.

Preguntas Probables de la Defensa

1. Por que elegiste Symfony y React?

Porque separan bien backend y frontend, tienen ecosistema maduro y permiten construir una API mantenible con una interfaz moderna y rapida para el usuario.

2. Por que has separado panel interno y portal externo?

Porque tienen usuarios, flujos y necesidades diferentes. Separarlos reduce complejidad, mejora la seguridad y hace mas clara la arquitectura.

3. Que problema real resuelve el proyecto?

Centraliza una gestion que antes estaba repartida entre correos, hojas de calculo y documentos, aportando trazabilidad y control del ciclo completo.

4. Cual es la funcionalidad mas diferencial?

El flujo completo de solicitud externa, verificacion, aprobacion interna y transformacion en empresa gestionable, junto con el workflow documental de convenios.

5. Por que usas SQLite en desarrollo?

Porque simplifica la puesta en marcha local y la demo. La arquitectura no queda atada a SQLite; Doctrine permite migrar a otros motores soportados.

6. Como has tratado la seguridad?

La API protege rutas internas con autenticacion y roles, y limita las rutas publicas al registro y verificacion de solicitudes. Como mejora futura queda sustituir Basic auth por un mecanismo mas robusto.

7. Que pruebas has realizado?

Pruebas backend automatizadas, builds de ambos frontends y verificacion funcional de los flujos principales durante la demo local.

8. Cual es la principal limitacion de la version actual?

La parte mas mejorable es la seguridad productiva y la automatizacion E2E. El proyecto esta bien para entorno academico y demostracion, pero no aun para explotacion real sin endurecimiento adicional.

9. Que aportacion personal destacarias?

El redisenio del alcance hacia un problema real del centro y la implementacion de una plataforma coherente de punta a punta: modelo, API, panel, portal y documentacion.

10. Que haria en una siguiente iteracion?

Refuerzo de autenticacion, politicas documentales, pruebas E2E, mejor refactor del frontend y salida PDF maquetada para informes y documentacion.

Checklist para la Defensa de Manana

Antes de salir

- llevar la memoria en PDF y el repositorio accesibles localmente;
- tener abierto `docs/guion-defensa.md` por si necesitas repasar el hilo;
- llevar tambien `docs/guia-demo.md` y `docs/preguntas-defensa.md`.

Antes de empezar la reunion

- comprobar que backend, panel y portal arrancan;
- dejar preparado el navegador en dashboard, convenios, solicitudes y portal;
- revisar que las credenciales demo estan visibles y correctas;
- tener una carpeta a mano con algun CSV exportado como evidencia.

Orden recomendado de presentacion

1. Problema real y objetivo.
2. Arquitectura en tres piezas.
3. Demo funcional.
4. Validacion tecnica.
5. Limitaciones y trabajo futuro.

Si algo falla en directo

- si falla un servicio, ensena la memoria, la guia de demo y los builds validados;
- si falla una ruta concreta, reconduce al flujo ya preparado en otra vista;
- si no da tiempo a toda la demo, prioriza dashboard, convenios, solicitudes y portal.

Cierre

- remarcar que el proyecto ya digitaliza el ciclo principal del centro;
- explicar brevemente que queda fuera por alcance, no por desconocimiento;
- terminar con una idea clara: trazabilidad, centralizacion y base escalable.

Documento generado automaticamente a partir de la memoria, anexos y material de defensa del proyecto.